

Un domeniu per serviciu — nginx în container

Exerciții · `compose-vhost.yaml` · nginx pe rețeaua compose

Primești `compose-vhost.yaml` : 8 containere. Unul singur publică un port — proxy-ul, pe 80. Restul nu au `ports`: deloc și nu se pot atinge de pe laptop.

Nu primești `vhost.conf`. Compose-ul îl montează, deci trebuie să existe *înainte* de primul `up`.

Data trecută rutai pe cale și fiecare aplicație trebuia convinsă să-și accepte prefixul. Aici rutezi pe **nume de gazdă**: aplicațiile stau la rădăcină, deci prefixul dispare cu tot cu problemele lui. Apar altele, mai interesante. Platforma cere aproximativ **200 MB de RAM**.

Contractul — ce trebuie să răspundă la final

Nume fără domeniu cumpărat: `nip.io` rezolvă orice `ceva.1.2.3.4.nip.io` către `1.2.3.4`. Înlocuiește `IP` cu IP-ul public al instanței.

Gazdă publică	Serviciu în compose	Port intern
who.IP.nip.io	whoami	80
status.IP.nip.io	status	3001
tools.IP.nip.io	tools	80
mail.IP.nip.io	mail	8025
logs.IP.nip.io	logs	8080
db.IP.nip.io	db-ui	8080

Porturile sunt cele **din container** — pe host nu mai e publicat niciunul. `postgres` nu primește gazdă. Orice alt nume primește un răspuns al tău, explicit.

Pasul 1 — Pornește platforma fără proxy

Instanță curată, Security Group cu 22 (My IP) și 80. Parola de Postgres se generează, nu se inventează. Pornești totul *în afară de proxy*.

CRITERIU DE GATA

```
docker compose -f compose-vhost.yaml config -q
docker compose -f compose-vhost.yaml up -d --scale proxy=0
docker compose -f compose-vhost.yaml ps
```

Coloana PORTS e goală la toate. Asta e și problema pasului 2.

Pasul 2 — Verifică servicii care n-au niciun port

`curl 127.0.0.1:8101` nu mai există aici: serviciile se văd doar dinăuntrul rețelei compose, după nume. Găsește singur cum ajungi acolo.

CRITERIU DE GATA

Un singur bloc de ieșire, cu cele 6 servicii și codul lor HTTP, obținut fără să publici niciun port nou și fără să intri în containerele aplicațiilor.

Pasul 3 — Prima gazdă

Scrie `vhost.conf` cu **o singură** gazdă, `who`, și pornește proxy-ul. Plus un bloc care prinde orice gazdă necunoscută și o spune pe față — nu vrei ca o greșeală de nume să cadă tăcut pe primul serviciu din fișier.

CRITERIU DE GATA — de pe laptop

```
curl -s http://who.IP.nip.io/  
curl -s http://habarnam.IP.nip.io/
```

Prima întoarce răspunsul de la whoami. A doua întoarce mesajul tău, nu whoami.

Pasul 4 — Toate cele șase

CRITERIU DE GATA — de pe laptop

```
for h in who status tools mail logs db; do  
  printf '%-7s -> ' $h  
  curl -s -o /dev/null -w '%{http_code}\n' -m 8 http://$h.IP.nip.io/  
done
```

Șase răspunsuri `200` sau `3xx`. Zero `502`.

Pasul 5 — Două dintre ele mint

CRITERIU — `200` nu înseamnă că merge

Deschide `status` și `mail` în browser, cu DevTools → Network, fila **WS**. Unul rămâne în „Connecting...”, celălalt nu află niciodată că a primit un mail. Amândouă au întors `200` la pasul 4, și amândouă au în Network o cerere oprită la `400`.

Află ce protocol cer, ce le lipsește când trec prin proxy, și repară.

CRITERIU DE GATA

În `status` ajungi la dashboard și poți adăuga un monitor. Cu interfața `mail` deschisă și neatinsă, trimiți un mail din interiorul rețelei și îl vezi apărând **fără refresh**:

```
printf 'Subject: proba\n\nlive\n' > m.txt  
docker run --rm --network vhost_default \  
  -v $PWD/m.txt:/m.txt:ro curlimages/curl:8.10.1 \  
  --url smtp://mail:1025 --mail-from a@lab.local \  
  --mail-rcpt b@lab.local --upload-file /m.txt
```

În Network → WS trebuie să existe o conexiune cu statusul `101`.

Pasul 6 — Rupe-l intenționat

Serviciile primesc IP de la Docker, iar IP-ul se schimbă la recreare. Forțează asta, cu proxy-ul rămas în picioare:

```
N=$(docker inspect vhost-whoami \  
  -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}')  
docker compose -f compose-vhost.yaml stop whoami  
docker run -d --name ocupant --network vhost_default --ip $N \  
  alpine:3.20 sleep 300  
docker compose -f compose-vhost.yaml up -d --force-recreate whoami
```

`ocupant` ține IP-ul vechi, deci `whoami` primește obligatoriu altul. Fără el, Docker dă adesea înapoi același IP și nu vezi nimic.

CRITERIU DE GATA

```
curl -s -o /dev/null -w '%{http_code}\n' http://who.IP.nip.io/
```

Scrie ce cod primești imediat, la 30 de secunde și la un minut, și ce se întâmplă după `docker exec vhost-proxy nginx -s reload`. Apoi explică în două fraze *când* citește nginx DNS-ul. Al doilea test, separat: oprește un serviciu și repornește proxy-ul. Citește ce scrie în `docker logs vhost-proxy` — e alt simptom, din aceeași cauză.

Pasul 7 — Repară-l ca să se repare singur

Fă ca nginx să nu mai țină minte adresa pentru totdeauna. Două lucruri lipsesc din `vhost.conf`, iar unul nu are efect fără celălalt.

CRITERIU DE GATA

Refaci exact reproducerea de la pasul 6. De data asta ruta se întoarce singură pe `200`, fără reload și fără restart, în câteva secunde. Scrii în predare cât a durat și de ce exact atât.

Ce predai

- `vhost.conf` — configurația ta, comisă în `lectii/L2-server-nginx/vhost-multi-domeniu/`
- `.env.example` — numele variabilelor, fără valori
- ieșirea de la pasul 2, codurile de la pașii 6 și 7, răspunsurile la provocări

Când ceva nu merge

Ce vezi	Unde te uiți întâi
host not found in upstream	serviciul nu rula în momentul în care a pornit nginx. Nu e greșeală de scriere
proxy-ul repornește la nesfârșit, iar în log scrie <code>is a directory</code>	<code>vhost.conf</code> nu exista la primul <code>up</code> , deci Docker a creat un director cu numele lui. Îl ștergi și pui fișierul
<code>502</code> pe o singură gazdă, restul merg	adresa ținută minte de la pornire. Pasul 6
toate gazdele duc în același loc	niciun <code>server_name</code> nu s-a potrivit, deci a răspuns blocul implicit
pagina se încarcă, dar nu se actualizează niciodată	DevTools → Network → WS. Pasul 5
browserul se învârt	Security Group. Timeout = firewall; refused = nu ascultă nimeni

Provocări

1. Oprește instanța EC2 și pornește-o din nou. IP-ul public se schimbă și toate cele șase `server_name` devin greșite deodată. Fă configurația să supraviețuiască schimbării de IP, fără să cumperi domeniu și fără să editezi fișierul după fiecare pornire.

2. `mail` ascultă și pe 1025, pentru SMTP. Scoate-l în lume prin nginx, pe portul 25. Atenție: ce scrii tu acum stă în `conf.d`, adică înăuntrul blocului `http` — iar SMTP nu e HTTP. Două fraze despre de ce nginx poate totuși, și de ce fișierul se montează în altă parte.

3. `logs` arată linii noi în timp real și a mers din prima, deși e tot o conexiune care nu se închide niciodată. De ce el da și celelalte două nu? Se vede într-un singur antet; pune-l în predare, lângă cel de la `status`.

4. Cere `http://IP/` direct, fără nume de gazdă. Cine răspunde și de ce tocmai el? Apoi răspunde la întrebarea care contează: dacă mâine cineva îndreaptă propriul lui domeniu către IP-ul tău, ce vede?